

Оптимальное управление в системах машинного обучения с петлёй обратной связи

Веприков Андрей Сергеевич
МФТИ, Москва
veprikov.as@phystech.edu

Хританков Антон Сергеевич
МФТИ, Москва
prog.autom@gmail.com

Аннотация

В работе рассматривается задача оптимального управления в системах машинного обучения (МО) с петлёй обратной связи. Под петлёй обратной связи мы понимаем ситуацию, когда предсказания модели на предыдущих шагах влияют на распределение данных, которое используется для обучения в будущем. В такой постановке состояние системы со временем становится зависимым от действий алгоритма МО.

Наша цель состоит в том, чтобы управлять процессом обучения так, чтобы в пределе получить желаемое распределение состояний системы, при этом сохранив саму петлю обратной связи и избегая деградации системы. Для математической формализации используется аппарат динамических систем и стохастического оптимального управления. Мы формулируем задачу в терминах (частично) наблюдаемого марковского процесса принятия решений, выделяем два ключевых оператора эволюции (данных и параметров модели), а затем предлагаем жадный алгоритм управления на основе анализа ω -предельного множества. На модельном примере показываем, что наивное жадное управление может привести к деградации системы, и обсуждаем модификации, позволяющие этого избежать. Отдельно рассматриваем связь с методами типа generative replay в постановках с несколькими задачами.

Ключевые слова: Стохастическое оптимальное управление, петля обратной связи, машинное обучение, generative replay.

1 Введение

Во многих современных приложениях машинного обучения модель не просто пассивно обрабатывает внешние данные, а активно влияет на среду, в которой эти данные генерируются. Типичные примеры — рекомендательные системы, динамическое ценообразование, системы ранжирования контента, персонализированные интерфейсы. В таких случаях предсказания модели на шаге $t - \tau$ меняют поведение пользователей и, как следствие, распределение данных, доступных для обучения на более поздних шагах t . Возникает *петля обратной связи* между моделью и данными.

Если игнорировать эту петлю, то можно получить систему, в которой краткосрочные метрики качества модели улучшаются, но долгосрочное поведение оказывается нежелательным: часть пользователей “выпадает” из системы, данные становятся всё менее разнообразными, а сама система может деградировать или “умирать”. Поэтому важно рассматривать не только задачу обучения модели на фиксированном датасете, но и задачу *управления* всей системой в динамике.

В данной работе мы рассматриваем абстрактную постановку такой задачи. Предполагается, что:

- система эволюционирует во времени $t = 0, 1, 2, \dots$;
- на каждом шаге существует распределение данных f_t и распределение параметров модели h_t ;
- доступно управление $u_t \in \mathcal{U}$, которое влияет только на процесс обучения модели (но не напрямую на данные);
- предсказания модели, обученной с применением управления u_t , влияют на дальнейшее распределение данных в системе.

Мы хотим построить такую стратегию управления $\{u_t\}_{t \geq 0}$, чтобы в пределе $t \rightarrow \infty$ система пришла к желаемому распределению состояний, а петля обратной связи не разрушилась. При этом сами переходы между состояниями сложны и задаются через два оператора эволюции:

1. оператор обновления параметров модели (обучение);

2. оператор обновления данных (реакция пользователей и среды на действия модели).

Для формализации этой задачи мы используем язык (Po)MDP, а затем уточняем постановку в терминах функций плотности, операторов эволюции и ограничений на траекторию системы. Далее мы вводим жадный алгоритм, который на каждом шаге подбирает управление, опираясь на анализ ω -предельного множества, и обсуждаем пример, иллюстрирующий необходимость учёта деградации системы. В заключительной части работы мы связываем предложенный подход с идеями generative replay в многозадачных постановках.

2 Постановка задачи в терминах (Po)MDP

Начнём с общей постановки задачи управления в терминах марковского процесса принятия решений (MDP). Рассмотрим пятёрку

$$(\mathcal{S}, \mathcal{U}, \mathbf{D}, r, \gamma),$$

где:

1. $\mathcal{S}$ — множество состояний системы;
2. $\mathcal{U}$ — множество допустимых действий (управлений);
3. $\mathbf{D}$ — оператор эволюции, задающий переходы между состояниями;
4. $r : \mathcal{S} \times \mathcal{U} \rightarrow \mathbb{R}$ — функция награды;
5. $0 < \gamma < 1$ — дисконтирующий коэффициент.

Классическая задача оптимального управления формулируется как

$$\max_{u_0, u_1, \dots} \sum_{t=0}^{+\infty} \gamma^t r(s_t, u_t) \quad (1)$$

при ограничении

$$s_{t+1} = \mathbf{D}(s_t, u_t). \quad (2)$$

В нашей задаче s_t представляет собой *сложное* состояние, включающее в себя описание текущего распределения данных и распределения параметров модели. При этом мы учитываем два важных отличия от стандартной MDP-постановки:

- на каждом шаге управление u_t может быть ограничено дополнительными условиями (например, сохранением петли обратной связи или недопустимостью деградации системы);
- оператор эволюции $\mathbf{D}$ естественно разложить на два компонента: обновление параметров модели и обновление распределения данных.

Далее мы уточним математическую структуру этих компонент.

3 Математическая постановка задачи

3.1 Состояния и операторы эволюции

Формализуем состояние системы и операторы эволюции.

1. Состояние системы на шаге t задаётся парой

$$(f_t, h_t) \in \mathcal{S},$$

где f_t — функция плотности (или более общий объект) для распределения данных, а h_t — функция плотности для распределения весов (параметров) модели МО.

2. На каждом шаге доступно управление

$$u_t \in \mathcal{U},$$

которое воздействует только на процесс обучения модели, а не напрямую на данные.

- 3(a). Оператор

$$\mathbf{P}_t \in \mathbf{P}$$

задаёт эволюцию распределения данных: $f_{t+1} = \mathbf{P}_t(f_t, h_{t+1})$. Его можно интерпретировать как “реакцию среды и пользователей” на текущую модель.

- 3(b). Оператор

$$\mathbf{H}_t \in \mathbf{H}$$

описывает обновление распределения параметров модели под действием данных f_t и управления u_t :

$$h_{t+1} = \mathbf{H}_t(f_t, h_t, u_t).$$

4. Функция награды

$$r(f_t, h_{t+1})$$

оценивает качество текущего состояния (или шага) системы. В зависимости от задачи она может учитывать качество предсказаний, близость к целевому распределению, стабильность системы и т.п.

5. Параметр $0 < \gamma < 1$ задаёт дисконтирование по времени и отвечает за баланс между краткосрочными и долгосрочными целями.

6. Важный элемент постановки — функция

$$g(f_t, h_{t+1}),$$

которая показывает, что в системе с данными $\sim f_t$ и весами модели $\sim h_{t+1}$ при нулевом дальнейшем управлении $u_{s \geq t} \equiv 0$ траектория $\{f_s, h_{s+1}\}_{s=t}^{\infty}$ является допустимой. Под допустимостью мы будем понимать, в частности, сохранение петли обратной связи и отсутствие деградации системы; точное определение может зависеть от конкретной прикладной задачи.

При этом часть информации о системе нам недоступна напрямую. Это удобно отразить в виде таблицы.

Недоступные объекты	Доступные объекты
$f_t \in \mathcal{S}$ $P_t \in \mathbf{P}$	Ограниченная выборка из f_t $H_t \in \mathbf{H}, u_t \in \mathcal{U}, g, \gamma$

Таблица 1: Наблюдаемость элементов системы.

3.2 Оптимизационная задача

Сформулируем задачу управления с учётом введённых обозначений. Мы хотим минимизировать суммарные дисконтированные потери:

$$\min_{u_0, u_1, \dots} \sum_{t=0}^{+\infty} \gamma^t r(f_t, h_{t+1}), \quad (3)$$

при ограничениях

$$h_{t+1} = H_t(f_t, h_t, u_t), \quad H_t \in \mathbf{H}, \quad (4)$$

$$g(f_t, h_{t+1}) \leq 0, \quad (5)$$

$$f_{t+1} = P_t(f_t, h_{t+1}), \quad P_t \in \mathbf{P}. \quad (6)$$

Ограничение (5) удобно интерпретировать как вероятностное условие:

$$\mathbb{P}\{g(\mathbf{x}, \boldsymbol{\theta}) \leq 0\} \geq 1 - \delta,$$

где $\mathbf{x} \sim f_t$, $\boldsymbol{\theta} \sim h_{t+1}$ и $0 < \delta < 1$ фиксировано. Эквивалентно,

$$\int_{\mathbf{x}} \int_{\boldsymbol{\theta}} \mathbf{1}\{g(\mathbf{x}, \boldsymbol{\theta}) \leq 0\} f_t(\mathbf{x}) h_{t+1}(\boldsymbol{\theta}) d\mathbf{x} d\boldsymbol{\theta} \geq 1 - \delta. \quad (7)$$

Таким образом, мы решаем задачу стохастического оптимального управления с функциональными состояниями и вероятностным ограничением на допустимость траектории.

4 Жадный алгоритм управления через ω -предельное множество

Прямая оптимизация задачи (3)–(6) в общем случае очень сложна. Вместо этого рассмотрим приближённый жадный алгоритм, который использует информацию об ω -предельном множестве системы.

Интуитивно ω -предельное множество для управления u_t описывает возможные предельные распределения состояний (f_s, h_{s+1}) при $s \rightarrow \infty$, если дальше управление не используется (то есть $u_{s>t} \equiv 0$). Тогда жадный шаг состоит в выборе такого u_t , которое приводит к наиболее желаемому ω -предельному состоянию.

Опишем алгоритм более формально.

4.1 Описание алгоритма

1. На шаге t нам доступна оценка $\hat{f}_t$ текущего распределения данных (через выборку) и текущее распределение параметров модели h_t , а также семейство операторов обучения $\mathbf{H}$.
2. Рассматриваем множество возможных управлений $u_t \in \mathcal{U}$ и отбрасываем те, для которых не выполняется ограничение (5). То есть сохраняются только такие u_t , при которых петля обратной связи не нарушается и система остаётся допустимой.
3. Для каждого оставшегося управления u_t рассматриваем динамику при $u_{s>t} \equiv 0$ и находим возможные предельные распределения параметров модели

$$\{h_{u_t}^\infty\},$$

которые могут быть достигнуты в пределе.

4. В задаче обучения с учителем, когда в системе есть петля обратной связи, предельное распределение данных $f_{u_t}^\infty$ имеет вид

$$f_{u_t}^\infty(\mathbf{x}, \mathbf{y}) > 0 \iff \mathbf{y} = m_{h_{u_t}^\infty}(\mathbf{x})$$

для всех состояний $(\mathbf{x}, \mathbf{y})$. То есть в пределе данные в системе согласованы с предсказаниями модели.

5. Среди допустимых u_t выбираем то, которое максимизирует (или минимизирует, в зависимости от задачи) некоторую целевую функцию $r(f_t, h_{t+1})$, отражающую желаемые свойства предельного распределения (например, близость к целевому множеству распределений).
6. Переходим к следующему шагу $t + 1$ и повторяем процедуру.

Такой жадный алгоритм не решает оптимизационную задачу (3) в полном объёме, но даёт понятный и интерпретируемый способ подбора управления в реальном времени.

5 Модельный пример для схемы на Рис. 1

В этом разделе мы рассмотрим упрощённый модельный пример, иллюстрирующий работу описанных операторов эволюции и ограничений. Схема эксперимента представлена на Рис. 1.

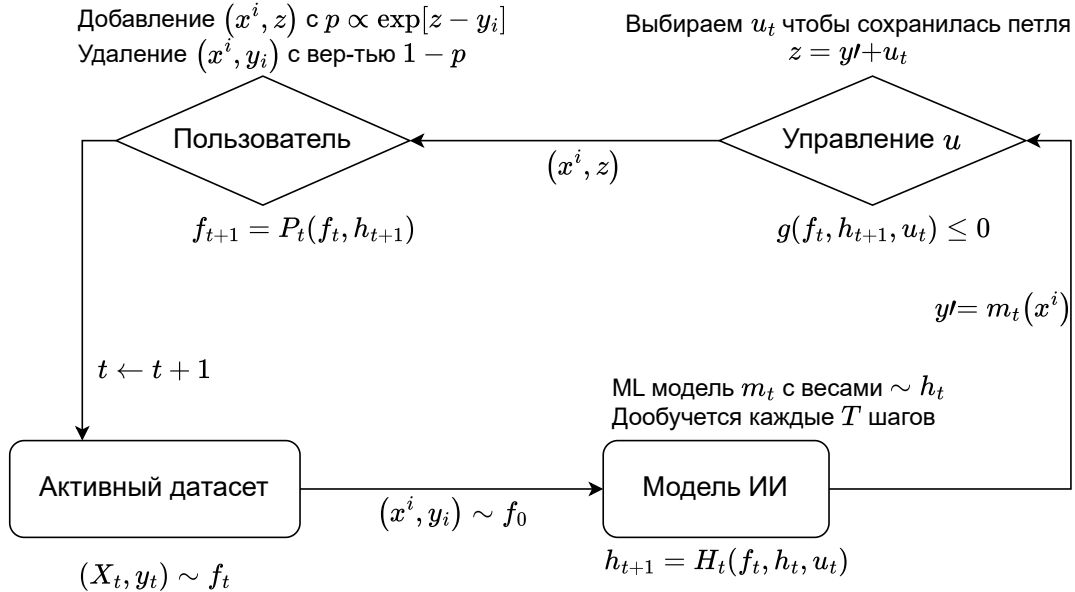


Рис. 1: Модельная постановка задачи оптимального управления в системе МО с петлёй обратной связи.

5.1 Эволюция распределения весов модели

Рассмотрим уравнение обновления распределения параметров:

$$h_{t+1} = H_t(f_t, h_t, u_t), \quad H_t \in \mathbf{H}.$$

В модельном эксперименте обучение происходит батчами: модель переобучается каждые T шагов, а между переобучениями параметры не меняются:

$$h_{t+1} = \begin{cases} h_t, & \text{если } t \text{ не делится на } T, \\ H(f_t, u_t), & \text{иначе,} \end{cases}$$

где $H(f_t, u_t)$ — оператор обучения модели $m_\theta(\cdot)$ по данным $\sim f_t$ с управлением u_t .

Один из естественных способов ввести управление u_t — рассматривать его как коэффициент регуляризации к некоторому множеству “хороших” весов Θ_t^F . Тогда h_{t+1} можно интерпретировать как распределение параметров θ^{t+1} , полученных из задачи

$$\theta^{t+1} \in \arg \min_{\theta \in \Theta} \left\{ \text{Loss}[m_\theta(\cdot), f_t] + \frac{u_t}{2} r(\theta, \Theta_t^F) \right\}, \quad (8)$$

где $r(\theta, \Theta_t^F)$ измеряет расстояние от вектора параметров θ до множества “желательных” параметров Θ_t^F .

5.2 Эволюция распределения данных

Уравнение для обновления распределения данных имеет вид

$$f_{t+1} = P_t(f_t, h_{t+1}), \quad P_t \in \mathbf{P}.$$

В простейшей модели среды предположим, что на каждом шаге t доступен датасет длины N , и:

- с вероятностью $p_t := \text{usage}_t$ мы заменяем один случайно выбранный элемент выборки предсказанием модели с шумом;
- с вероятностью $1 - p_t$ оставляем выборку без изменений.

Формализуем это. Пусть $\xi \sim \text{Be}(p_t)$ — индикатор замены, и заменяем только один из N элементов. Тогда по формуле полной вероятности

$$\begin{aligned} f_{t+1}(\cdot) &= \frac{1}{N} [\mathbb{P}[\xi = 0] f_{t+1}(\cdot \mid \xi = 0) + \mathbb{P}[\xi = 1] f_{t+1}(\cdot \mid \xi = 1)] + \left(1 - \frac{1}{N}\right) f_t(\cdot) \\ &= \left(\frac{1 - p_t}{N} + \frac{N - 1}{N}\right) f_t(\cdot) + \frac{p_t}{N} G_t(h_{t+1}, f_t)(\cdot) \\ &= \left(1 - \frac{p_t}{N}\right) f_t(\cdot) + \frac{p_t}{N} G_t(h_{t+1}, f_t)(\cdot), \end{aligned} \quad (9)$$

где $G_t(h_{t+1}, f_t)$ — оператор, который из распределения параметров модели h_{t+1} и данных f_t строит новое распределение данных (после применения действий модели и шума).

В рассматриваемом эксперименте данные обновляются за счёт добавления шума к предсказаниям модели:

$$G_t(h_{t+1}, f_t) = \text{свёртка} \left[\begin{pmatrix} m_{\theta^{t+1}}(\mathbf{x}) \\ \mathbf{x} \end{pmatrix}; \begin{pmatrix} \mathcal{N}(0, \text{adherence}_t^2) \\ \text{шум по } \mathbf{x} \end{pmatrix} \right], \quad (10)$$

где $\mathbf{x} \sim f_t$, $\theta^{t+1} \sim h_{t+1}$, а параметр adherence_t задаёт уровень шума в целевой переменной.

5.3 Условие сохранения петли обратной связи

Ограничение

$$g(f_t, h_{t+1}) \leq 0$$

гарантирует, что выбранное управление не разрушает петлю обратной связи и не приводит к нежелательному поведению системы.

Интуитивно можно считать, что допустимыми являются такие пары (f_t, h_{t+1}) , при которых:

- модель демонстрирует достаточно высокое качество на текущих данных;
- система не деградирует (не теряет пользователей, не вырождается распределение данных и т.п.);
- в дальнейшей динамике (при отсутствии дополнительного управления) система остаётся в “здоровом” режиме работы.

Функция g может включать в себя несколько компонент. Например, одна из компонент может моделировать сохранение петли обратной связи через требование, чтобы качество модели на текущем распределении данных не опускалось ниже заданного порога: иначе пользователи перестанут реагировать на рекомендации, и петля разрушится.

Сделаем одно естественное предположение.

Предположение 1 (Начальная допустимость). Для начальных распределений данных f_0 и параметров модели h_0 выполнено $g(f_0, h_0) \leq 0$.

Это предположение аналогично выбору начальной точки из допустимого множества в задачах оптимизации с ограничениями. Оно гарантирует (по индукции), что при выборе $u_t = 0$ множество допустимых управлений на каждом шаге не пусто.

6 Итоговый алгоритм и учёт деградации системы

6.1 Жадный алгоритм с упрощёнными предположениями

Сформулируем итоговый жадный алгоритм для модельной системы, опираясь на сделанные выше допущения.

Предположение 2. *Выполнены следующие условия:*

1. Если $g(f_t, h_{t+1}) \leq 0$, то интенсивность использования модели usage_t постоянна (зависит от f_t и h_{t+1} по типу барьерной функции). При этом при $u_t = 0$ обязательно выполняется $g(f_t, h_{t+1}) \leq 0$.
2. Параметр adherence_t постоянен и не зависит от f_t и h_{t+1} .

Тогда из (10) следует, что управление u_t влияет на оператор G_t только через модель $m_{\theta^{t+1}}$. Это значит, что мы можем менять *только* распределение целевой переменной y в системе (при фиксированном распределении признаков). Введём целевое множество распределений по y :

$$\mathcal{F}|_y,$$

полученное как “сечение” некоторого желаемого множества распределений $\mathcal{F}$.

Из (9) видно, что при $p_t = 0$ или постоянном p_t мы можем управлять только близостью распределения $G_t(h_{t+1}, f_t)$ к $\mathcal{F}|_y$. Это достигается выбором параметров модели θ^{t+1} , чьи предсказания по y при $\mathbf{x} \sim f_t$ дают желаемое распределение.

Обозначим через $\Theta_t^{\mathcal{F}}$ множество “идеальных” параметров, для которых данные вида $(m_{\theta}(\mathbf{x}^t), \mathbf{x})^{\top}$, $\mathbf{x} \sim f_t$ попадают в $\mathcal{F}$. Тогда жадный алгоритм можно описать так.

Итоговый жадный алгоритм.

1. На шаге t доступен датасет

$$\{(x^i, y_i)\}_{i=1}^n.$$

2. Вычисляем множество желательных параметров:

$$\Theta_t^{\mathcal{F}} = \arg \max_{\theta \in \Theta} \left\{ \sum_{i=1}^n \mathbf{1}[m_{\theta}(x^i) \in \mathcal{F}|_y] \right\}.$$

В простом случае можно предполагать, что $\Theta_t^{\mathcal{F}}$ не зависит от t , то есть желаемые веса модели известны заранее.

3. Выбираем достаточно малое $u_t \in \mathbb{R}_+$, которое задаёт силу регуляризации в (8).
4. Обучаем модель, решая задачу

$$\theta^{t+1} \in \arg \min_{\theta \in \Theta} \left\{ \text{Loss}[m_{\theta}(\cdot), f_t] + \frac{u_t}{2} r(\theta, \Theta_t^{\mathcal{F}}) \right\}.$$

5. Если метрики качества модели $m_{\theta^{t+1}}$ на данных f_t ниже заданного порога $1 - \delta_t$, уменьшаем $u_t \leftarrow u_t/2$ и повторяем обучение. Иначе переходим к следующему шагу $t + 1$.

6.2 Деградация системы

На практике важно не только сохранять петлю обратной связи, но и избегать *деградации системы*. Под деградацией будем понимать ситуацию, когда число активных пользователей или разнообразие данных в системе постепенно уменьшается, даже если качество модели по стандартным метрикам растёт.

Простой пример — динамическое ценообразование. Пусть модель постепенно повышает цены на товары, стремясь максимизировать выручку. При этом с каждым шагом часть пользователей отказывается от покупок и покидает систему. Через некоторое время может оказаться, что даже при высоких метриках качества на оставшихся данных, система фактически “умирает”: новых сделок почти нет, а активных пользователей очень мало.

Один из способов формализовать деградацию — через “уверенность” в оценке распределения данных. Пусть $\hat{f}_t$ — некоторая статистическая оценка f_t . Чем точнее мы можем её оценить (чем больше выборка), тем “живее” система. Тогда деградация может быть связана с ростом расстояния $\|\hat{f}_t - f_t\|^2$.

Практически удобнее отслеживать размер активного датасета. Пусть N_t — число элементов в датасете, собранном на шаге t (или на окне шагов, включающих t). Тогда:

- если $N_t \geq N_{t-1}$ или $N_t/N_{t-1} \geq 1 + \delta$, система не приближается к смерти;
- если $N_t \rightarrow 0$, система деградирует, и стратегию управления нужно менять.

На Рис. 2 показан модельный эксперимент, в котором при наивном жадном управлении система деградирует, хотя метрики качества модели растут. Проблему можно решить, модифицировав функцию g в (5), добавив в неё явную зависимость от динамики f_t (например, через N_t или $\|\hat{f}_t - f_t\|$).

Идея состоит в следующем:

- если отклонение между f_t и его оценкой $\hat{f}_t$ невелико, система стабильна, и жадное управление допустимо;

- если отклонение велико, система близка к деградации, и нужно ослабить управление или изменить его направление, чтобы сохранить “живость” данных.

В численных экспериментах такой подход позволяет одновременно:

- избегать деградации (сохранить достаточное количество активных пользователей/объектов);
- постепенно приближать распределение данных к целевому $\mathcal{F}$.

Вычислительный эксперимент. Для иллюстрации в работе рассматривается следующая схема. На каждом шаге t имеется активный датасет размера N_t с признаками объектов недвижимости X_t и их ценами y_t . Обновление данных происходит так:

1. случайно выбирается индекс $i \sim U(1, N_0)$;
2. вычисляется предсказание модели: $z = m_t(X_0[i]) + u_t$, где m_t — модель, дообучаемая каждые T шагов, а u_t — управление (сдвиг цены);
3. исходная цена $y_0[i]$ заменяется на z , и пара $(X_0[i], z)$ добавляется в активный датасет с вероятностью $p \propto \exp[-(z - y_0[i])]$;
4. с вероятностью $1 - p$ объект i удаляется из активного набора.

Такой механизм моделирует поведение пользователей: чем выше цена относительно исходной, тем меньше вероятность покупки и тем выше шанс “выпадения” пользователя. Целью управления является повышение цен как можно сильнее, при этом не “убив” систему.

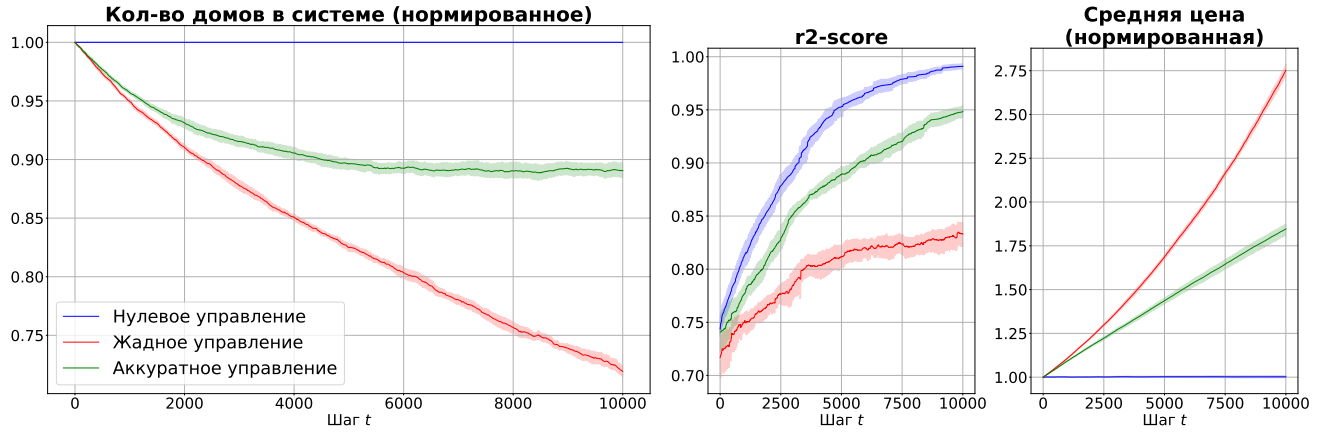


Рис. 2: Модельный эксперимент: наивное жадное управление ведёт к деградации системы (слева — динамика размера активного датасета, справа — качество модели).

Как видно из рисунка, без явного учёта деградации в функции g жадное управление способно существенно ухудшить состояние системы, несмотря на рост качества модели по стандартным метрикам.

7 Связь с Generative Replay в многозадачных постановках

В многозадачном обучении (multi-task или continual learning) популярны методы *generative replay*. Основная идея состоит в том, что при обучении на новой задаче модель дополнительно использует сгенерированные примеры, имитирующие данные прошлых задач. Это позволяет смягчить катастрофическое забывание.

Классическая форма записи функции потерь на шаге $t + 1$:

$$L_{\text{train}}(\theta_{t+1}) = u \underbrace{\mathbb{E}_{(\mathbf{x}, \mathbf{y}) \sim D_t} [L(S(\mathbf{x}; \theta_{t+1}), \mathbf{y})]}_{\text{потери на новой задаче}} + (1 - u) \underbrace{\mathbb{E}_{\mathbf{x}' \sim G_{t-1}} [L(S(\mathbf{x}'; \theta_{t+1}), S(\mathbf{x}'; \theta_t))]}_{\text{потери на "старых" задачах через генератор}},$$

где:

- D_t — датасет текущей задачи;
- G_{t-1} — генератор, имитирующий данные прошлых задач;
- $S(\cdot; \theta)$ — основная модель;

- $u \in [0, 1]$ — параметр, задающий баланс между новой и старой информацией.

Эту постановку можно интерпретировать в той же общей схеме: распределение данных на шаге t есть смесь текущего датасета и сгенерированных данных, а управление u_t выбирает баланс между ними. Тогда:

- роль f_t играет эффективное распределение данных $\mathcal{D}_t$ (смесь D_t и G_{t-1});
- роль u_t играет коэффициент u , определяющий вероятность взять новый или сгенерированный пример;
- функция g может задавать минимальный уровень качества на старых задачах, запрещая такие значения u , при которых забывание становится слишком сильным.

7.1 Постановка через скользящее окно

Удобно переформулировать generative replay через *скользящее окно* длины $|\mathcal{D}_t|$. Пусть есть n задач, суммарная длина всех датасетов равна $T = \sum_{i=1}^n |D_i|$, и на первом шаге $\mathcal{D}_1 = D_1$. Далее для каждого шага $t \in [1; T]$ выполняем:

- удаляем первый элемент $\mathcal{D}_t[0]$ (самый старый);
- с вероятностью u_t выбираем $(\mathbf{x}, \mathbf{y}) \sim D_{t//n}$ (пример из текущего датасета);
- с вероятностью $1 - u_t$ выбираем $\mathbf{x} \sim G_{t//n}$ и $\mathbf{y} = S_{t//n}(\mathbf{x})$ (сгенерированный пример с “таргетом” как предсказание старой модели);
- добавляем выбранную пару $(\mathbf{x}, \mathbf{y})$ в конец окна $\mathcal{D}_t$;
- если $t \bmod n = 0$, дообучаем пару (S_t, G_t) на окне $\mathcal{D}_t$.

Таким образом, u_t становится управлением, определяющим, как быстро система “забывает” старые задачи, и как сильно в $\mathcal{D}_t$ доминируют новые данные.

7.2 Примеры расписаний для u_t

Рассмотрим несколько простых расписаний:

- $u_t = \frac{1}{(t//n) + 1}$. Тогда для каждой задачи r на шаге обучения ожидается одинаковое число примеров от задач $1, \dots, r$.
- $u_t = u_0 = \text{const}$ — постоянное соотношение новых и старых данных.
- $u_t = \exp\left(\frac{t}{T}\right)$ или убывающие/возрастающие экспоненты.
- $u_t = 1 - u_0 t^\alpha$ или $u_t = 1 - \frac{u_0}{t^\alpha + 1}$ — степенные расписания.

С точки зрения предложенной в работе общей постановки, выбор расписания u_t — это выбор стратегии управления $\{u_t\}$, влияющей на предельное распределение данных и параметров модели.

7.3 Численные эксперименты

Парабола. В первом эксперименте мы рассматриваем регрессию для функции $y = x^2$. Интервал значений x разбивается на n подотрезков, и для каждого подотрезка формируется отдельный датасет:

$$D_r = \{(x_i^r, y_i^r = (x_i^r)^2)\}_{i=1}^{T//n}, \quad x_i^r \sim \text{Uniform}[r, r+1],$$

для всех $r \in \{1, \dots, n\}$. В качестве модели используется линейная регрессия, обучаемая SGD. В таком эксперименте идеальный нулевой лосс недостижим (данные квадратичны, модель линейна), зато хорошо видно влияние выбора u_t на форму приближения.

На Рис. 3 приведены типичные результаты для различных расписаний u_t .

SVD. Во втором эксперименте строится матрица $X \in \mathbb{R}^{(nN) \times n}$ с полным рангом, выполняется её SVD-разложение

$$X = \sum_{r=1}^n \sigma_r u_r v_r^\top,$$

и для каждой задачи r задаётся датасет

$$D_r = (X^r = \sigma_r u_r v_r^\top, Y^r = \sigma_r u_r).$$

Нетрудно проверить, что нулевой лосс на задаче r достигается при любом векторе

$$w = v_r + \sum_{r' \neq r} \alpha_{r'} v_{r'}$$

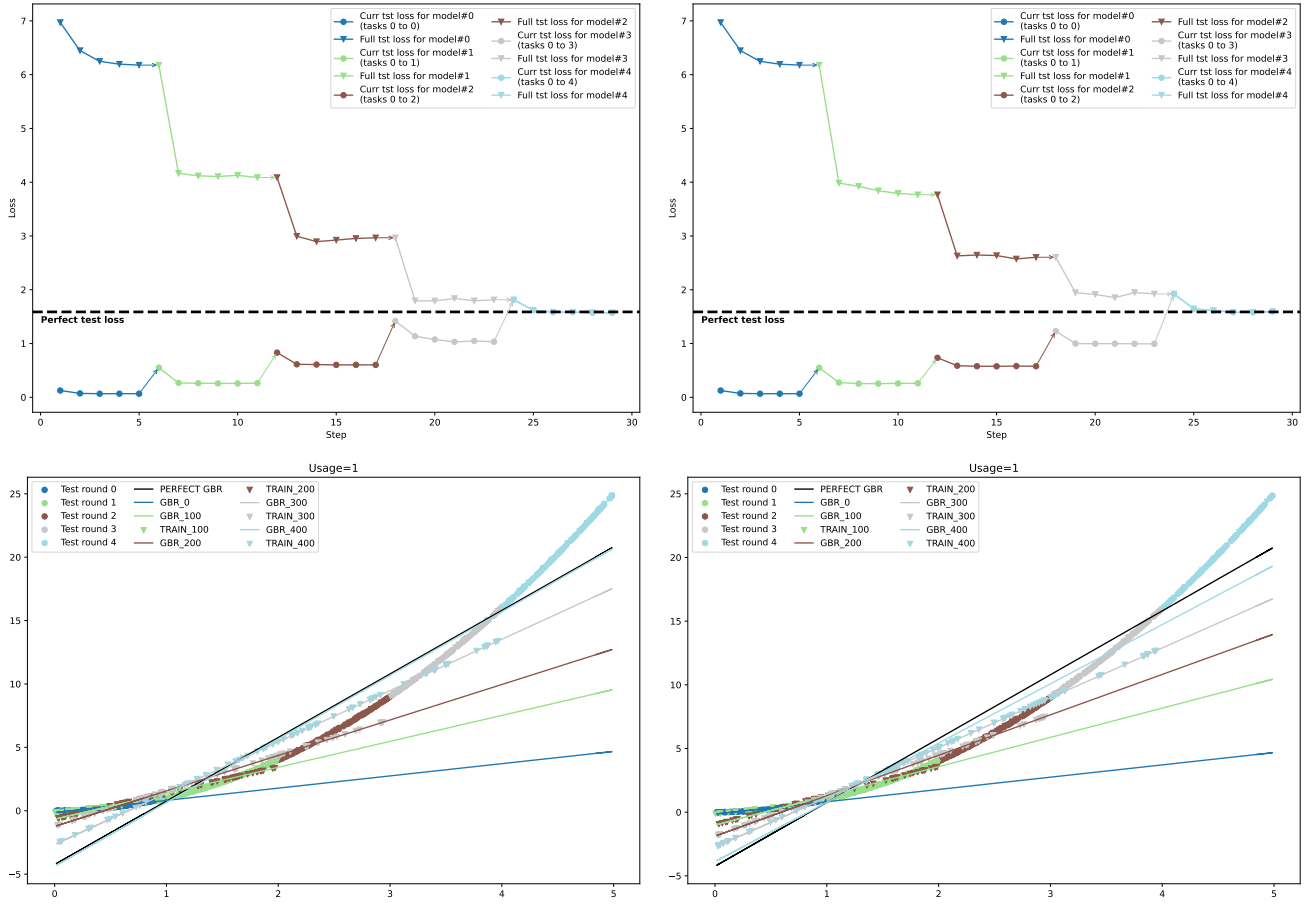


Рис. 3: Результаты эксперимента с параболой. Слева направо: инверсное расписание u_t и экспоненциальное.

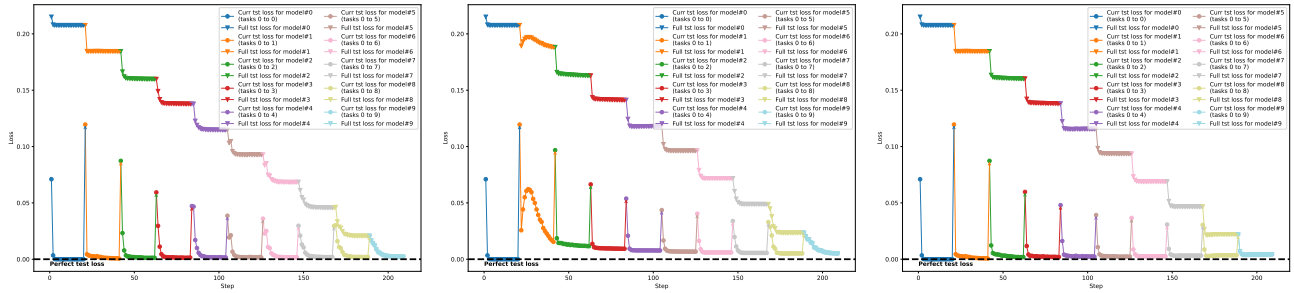


Рис. 4: Результаты SVD-эксперимента. Слева направо: инверсное, экспоненциальное и постоянное расписания u_t .

для произвольных α_r , а нулевой лосс на всех задачах одновременно — при $w = \sum_{r=1}^n v_r$.

Разные расписания u_t по-разному влияют на то, к какому вектору w в итоге стремится обучение. На Рис. 4 показаны примеры динамики лоссов для инверсного, экспоненциального и постоянного расписаний.

Эти эксперименты иллюстрируют, что даже в простых задачах выбор стратегии управления u_t существенно влияет на предельное состояние системы (параметры модели и распределение данных).

8 Заключение

В работе предложена общая постановка задачи оптимального управления в системах машинного обучения с петлёй обратной связи. Вместо того чтобы рассматривать обучение модели на фиксированном датасете, мы моделируем совместную эволюцию распределения данных и распределения параметров модели, а также влияние управления на

процесс обучения. Это позволяет:

- учитывать долгосрочные эффекты предсказаний модели на данные;
- формулировать цели в терминах желаемого предельного распределения состояний;
- явно накладывать ограничения на сохранение петли обратной связи и недопущение деградации системы.

Были введены операторы эволюции для данных и параметров модели, а также функция g , задающая допустимость траектории. На этой основе сформулирована задача стохастического оптимального управления и предложен жадный алгоритм, использующий идею ω -предельного множества. На модельных примерах показано, что:

- наивное жадное управление может приводить к деградации системы, даже если качества модели по стандартным метрикам растёт;
- введение дополнительных условий в функцию g , отслеживающих состояние системы (например, размер активного датасета), позволяет избежать деградации;
- популярные методы вроде generative replay естественным образом укладываются в предложенную схему как частный случай управления распределением данных через параметр u_t .

Перспективные направления дальнейшей работы включают:

- более строгий анализ сходимости и устойчивости жадных стратегий управления;
- разработку практических критериев для функции g , отражающих реальные ограничения прикладных систем (ограничения по справедливости, устойчивости, разнообразию данных и т.п.);
- расширение постановки на случай частично наблюдаемых состояний (PoMDP) и обучение стратегий управления с помощью методов обучения с подкреплением.